

PROJECT REPORT

Project 2: Basic Encryption & Decryption

Implementing the Caesar Cipher for Data Confidentiality

Goal	Implement a simple encryption and decryption technique using a Caesar cipher.
Key Skills	Encryption concepts, logic building, data protection basics
Language Used	Python 3
Status	Complete — all deliverables implemented and validated

1. Objective

The goal of this project was to implement a basic encryption and decryption technique the Caesar cipher that transforms readable text (plaintext) into unreadable text (ciphertext), and reverses the process to recover the original message.

Key Requirements

- Encrypt user text using a basic logic (Caesar cipher)
- Decrypt the encrypted text
- Display both encrypted and decrypted output
- Handle edge cases: spaces, punctuation, and letter case
- Validate correctness: decrypting the ciphertext must return the original plaintext

2. Conceptual Framework: The IPO Model

Every cryptographic system follows the universal Input → Process → Output architecture:

Stage	Description
Input	Plaintext (raw data) supplied by the user
Process	Algorithm + Key — the mathematical transformation applied to the input
Output	Ciphertext (secured) — the unreadable result of the process

The core insight behind this model is: "We don't shift letters; we shift integers." Text must first be converted into numbers before any mathematical operation can be applied to it.

2.1 Input Logic: ASCII Conversion

Every character has a fixed numeric (ASCII) value. For example, the letter 'A' has the ASCII value 65. Python's `ord()` function converts a character to its ASCII integer, and `chr()` reverses this, converting an integer back to a character.

Character	ASCII Value
A ... Z	65 ... 90
a ... z	97 ... 122

2.2 Process Logic: The Shift

The Caesar cipher is a mono-alphabetic substitution cipher: each letter is replaced by another letter a fixed number of positions ('shift' or 'key') further along the alphabet.

$$E_n(x) = (x + n) \% 26$$

- x = character position in the alphabet (A = 0 ... Z = 25)
- n = the key (shift amount)
- $\% 26$ = modular arithmetic that wraps the result around the 26-letter alphabet

2.3 The Math of the Wrap

Modular arithmetic is essential because the alphabet is finite — without it, shifting near the end (e.g., 'X', 'Y', 'Z') would produce invalid values. Worked example, shifting 'Y' by 3:

```
Input Y (position 24) + Shift (3) = 27
27 % 26 = 1
Position 1 = 'B'

Result: Y --> B
```

2.4 Reverse Engineering: Decryption Logic

Because the Caesar cipher is symmetric, decryption simply reverses the shift using the same key:

$$D_n(x) = (x - n) \% 26$$

Ciphertext → Reverse Shift (−Key) → Plaintext. "The same key locks and unlocks" this is what makes the Caesar cipher a symmetric-key algorithm.

3. Algorithm Design

Step-by-step data flow used for each character, shown for the letter 'A' with shift = 3:

```
Char 'A'
| ASCII Conversion      -> 65
| Subtract Base (-65)   -> 0
| Add Key (+3)          -> 3
| Modulo (% 26)         -> 3
| Add Base (+65)        -> 68
v
Char 'D'
```

Edge Case Handling

- Uppercase letters (A-Z) are shifted using base 65
- Lowercase letters (a-z) are shifted using base 97, so case is preserved
- Non-alphabetic characters spaces, punctuation, digits are passed through unchanged, since they fall outside the A-Z / a-z ranges
- Negative shifts (used for decryption) are handled correctly because Python's % operator always returns a non-negative result for a positive modulus

4. Implementation (Python)

Translating the math directly into code using ord() and chr():

```
def encrypt(text: str, shift: int) -> str:
    result = []
    for char in text:
        if char.isupper():
            shifted = (ord(char) - 65 + shift) % 26 + 65
            result.append(chr(shifted))
        elif char.islower():
            shifted = (ord(char) - 97 + shift) % 26 + 97
            result.append(chr(shifted))
        else:
            result.append(char) # spaces, punctuation, digits
    return "".join(result)

def decrypt(cipher: str, shift: int) -> str:
    return encrypt(cipher, -shift) # same key, reversed shift
```

Key Functions used:

- ord() — Character → Integer
- chr() — Integer → Character
- % 26 — keeps the result within the bounds of the alphabet

5. Testing & Validation

The program was tested with a shift key of 3 (and other values) against inputs containing letters, spaces, punctuation, digits, and case variation. Each test confirms `decrypt(encrypt(x)) == x`.

Plaintext	Encrypted → Decrypted
-----------	-----------------------

Hello, World! 123	Khoor, Zruog! 123 → Hello, World! 123
attack at dawn	fyyfhp fy ifbs → attack at dawn
XYZ xyz	BCD bcd → XYZ xyz

Result: All test cases passed — every decrypted output exactly matched the original plaintext, and letter-wrapping at the end of the alphabet (X→A, Y→B, Z→C) worked correctly.

6. Security Analysis: Why the Caesar Cipher Is a Lockbox, Not a Vault

While functionally correct, the Caesar cipher is cryptographically weak by modern standards, for two key reasons:

- Tiny key space Only 25 possible shifts exist, meaning brute force can try every option almost instantly.
- Pattern preservation The cipher is a 1-to-1 substitution, so the frequency distribution of letters in the ciphertext is identical in shape to normal English letter frequency, just shifted. This makes the cipher breakable via frequency analysis, without even knowing the key.

This vulnerability is the motivation for modern cryptography, which layers many mathematical operations together instead of a single shift.

The Evolution: From Caesar to AES

Caesar Cipher → Confusion + Diffusion → AES (128-bit keys, XOR operations). Modern algorithms like AES combine substitution and complex transformations across many rounds, using much larger key spaces, making brute-force and frequency analysis infeasible.

7. Conclusion

This project successfully implements the full IPO cycle for basic encryption: converting user text into ciphertext via a mathematical shift, and reversing that transformation to recover the original message. All required deliverables were met:

- Implemented the IPO Cycle (Input → Process → Output)
- Applied the math: `ord()`, `chr()`, `% 26`
- Handled edge cases (spaces, punctuation, case)
- Validated correctness with a decryption function

Mastering the logic of the shift lays the mathematical foundation for understanding stronger, modern encryption systems used to protect data in transit today.